Etapa 1: Mockup al interfeței (HTML și CSS)

Tehnologii Web · Proiect de semestru · Etapa 1

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Inginerie Industrială și Robotică
Disciplina: Tehnologii Web, proiect de semestru
Anul universitar: 2026–2027
Coordonatori de proiect: Emilia PĂSĂRIN, Alexandru RIZEA, Andrei-Alexandru STAICU

Enunțul etapei

Să se aleagă tema proiectului de semestru și să se realizeze, numai cu HTML și CSS, prima versiune a interfeței aplicației: un antet, un formular de adăugare și o listă de trei carduri completate cu datele temei. Pagina trebuie să se rearanjeze pe ecrane înguste, să aibă focus vizibil la navigarea cu tastatura și o temă întunecată. Codul se publică într-un repository GitHub personal, însoțit de un fișier README care descrie tema.

Etapa nu presupune cunoștințe anterioare de JavaScript. Toate datele afișate în pagină se scriu, deocamdată, direct în HTML; începând cu Etapa 2, ele vor fi generate din cod.

La finalul etapei veți putea:

- descrie o idee de aplicație printr-un model de date simplu;
- construi structura unei pagini cu elemente HTML semantice;
- așeza elementele în pagină cu Grid și Flexbox;
- adapta pagina la ecrane înguste printr-o regulă `@media`;
- defini culorile prin variabile CSS și obține din ele o temă întunecată;
- crea un repository Git, salva versiuni ale codului și publica aceste versiuni pe GitHub.

Organizarea proiectului

Punctaj și prezență

Proiectul valorează 30 de puncte. Se susțin trei evaluări a câte 10 puncte: în **săptămâna 6**, în **săptămâna 9** și la **evaluarea finală**. În celelalte săptămâni, progresul este verificat la fiecare etapă, fără notare. **Prezența la toate etapele de proiect este obligatorie.**

La fiecare evaluare, studentul explică o secțiune din codul propriu sau efectuează pe loc o modificare simplă. Verificarea confirmă înțelegerea codului predat, indiferent dacă acesta a fost scris integral de student sau cu ajutorul unui instrument AI.

Etapele proiectului

Pe parcursul semestrului, fiecare student dezvoltă o singură aplicație web, construită incremental: fiecare etapă extinde rezultatul etapei anterioare, iar codul scris într-o etapă se păstrează în etapele următoare (tabelul 1.1).

Tabelul 1.1: Primele etape ale proiectului

Săptămâna	Conținut	Livrabil
1	Mockup al interfeței (HTML și CSS)	pagina statică pe GitHub, cu README
2	Logica pe date (JavaScript)	funcții care operează pe o listă de obiecte
3	Inițializarea proiectului React (Vite)	aplicația React funcțională, cu datele temei
4	Componente construite din date	lista afișată prin componente
5–13	interacțiune, API, formulare, server, bază de date, autentificare, Docker	aplicația completă și prezentarea finală

Materialele folosesc ca exemplu aplicația **TaskFlow**, o listă de sarcini, prezentă în toate figurile. Exemplul are exclusiv rol de referință: fiecare student realizează propria aplicație, pe propria temă. O aplicație identică cu TaskFlow, doar redenumită, nu se punctează.

Predarea și verificarea

Proiectul se predă **exclusiv prin repository-ul GitHub al studentului**, public și același pe tot parcursul semestrului. Nu se trimit fișiere prin e-mail sau prin alte platforme. Se verifică numai conținutul publicat pe GitHub: un commit realizat local, fără `git push`, nu este vizibil.

La fiecare etapă se realizează cel puțin un commit, cu mesaj de forma `Stage N: ...`. Istoricul commit-urilor arată cum a evoluat proiectul de la o etapă la alta.

Fiecare cerință a unei etape are un cod (de exemplu **S1-R4**). La finalul etapei, README-ul conține un **tabel de verificare** în care studentul indică, pentru fiecare cerință, locul exact din cod unde este îndeplinită și modul în care se poate verifica. Modul de completare este descris în secțiunea *Tabelul de verificare din README*.

Utilizarea instrumentelor AI

Utilizarea instrumentelor AI (ChatGPT, Claude, Copilot, Gemini etc.) este permisă, cu condiția declarării în scris a **instrumentelor folosite și a scopului în care au fost folosite**. Dacă s-au utilizat mai multe instrumente, se declară toate. Declararea are două componente:

- **în README**, secțiunea *AI usage*: un rezumat pentru întregul proiect, cu fiecare instrument și scopul utilizării;
- **un jurnal pentru fiecare etapă**, în folderul `ai-log/` al repository-ului (`etapa-01.md`, `etapa-02.md` etc.), care conține instrumentele utilizate, link-urile de partajare ale conversațiilor (opțiunea *Share* din ChatGPT, Claude sau Gemini), 2–3 solicitări relevante cu răspunsul primit și modificările aduse, precum și câteva rânduri despre ce s-a învățat.

Jurnalul nu presupune copierea integrală a conversațiilor. Pentru sugestiile automate din editor (de exemplu Copilot) se menționează doar utilizarea lor și fișierele în care au fost folosite. Dacă într-o etapă nu s-a folosit niciun instrument AI, fișierul conține un singur rând care precizează acest lucru.

Alegerea temei

Tema este la alegerea studentului, cu o singură condiție: aplicația trebuie să gestioneze **o listă de elemente**, la fel ca TaskFlow. Etapele sunt comune pentru toți studenții și sunt concepute pentru acest tip de aplicație. Fiecare element din listă trebuie să conțină cel puțin cele cinci informații din tabelul 1.2.

Tabelul 1.2: Informațiile obligatorii ale unui element

Informația	La TaskFlow	La o bibliotecă de cărți	Etapa în care devine necesară
un nume, introdus de utilizator	titlul sarcinii	titlul cărții	2
o stare de tip da/nu, care poate fi bifată	finalizată sau nu	citită sau nu	2
o etichetă aleasă dintr-o listă fixă	prioritate: mică, medie, mare	format: hârtie, ebook, audio	2
o categorie	Facultate, Personal, Muncă	Roman, SF, Istorie	10
utilizatorul căruia îi aparține	autorul sarcinii	proprietarul cărții	11

Absența uneia dintre aceste informații blochează etapele ulterioare: fără starea da/nu nu există ce bifa în Etapa 5, iar fără utilizator nu se pot separa datele la autentificare. Se pot adăuga oricâte câmpuri suplimentare (autorul unei cărți, durata unei rețete etc.).

Exemple de teme potrivite: o listă de filme văzute sau de văzut, o colecție de rețete, o colecție de jocuri, un jurnal de antrenamente, evidența echipamentelor dintr-un laborator. Sunt acceptate și teme propuse de student, dacă respectă condițiile de mai sus. **Tema se aprobă de coordonator în cadrul acestei etape**, pe baza fișierului README; fără temă aprobată, etapa nu este încheiată.

Adaptarea exemplului la tema proprie

Exemplele de cod și figurile provin din TaskFlow și nu se copiază ca atare: fiecare element specific TaskFlow se înlocuiește cu echivalentul său din tema proprie. De exemplu, pentru o bibliotecă de cărți numită BookShelf, textul „Prioritate: mică, medie, mare” devine „Format: hârtie, ebook, audio”, iar stările „Activ / Gata” devin „Necitită / Citită”. Ultima coloană a tabelului 1.3 se completează înainte de începerea implementării.

Tabelul 1.3: Corespondența dintre exemplu și tema proprie

Element al paginii	Exemplu (TaskFlow)	Exemplu: BookShelf	Tema proprie
numele aplicației, în antet	TaskFlow	BookShelf	
câmpul text din formular	Titlu	Titlu carte	
lista fixă din formular	Prioritate: mică, medie, mare	Format: hârtie, ebook, audio	
cele două stări afișate pe card	Activ / Gata	Necitită / Citită	
clasele CSS pentru etichete	badge-mica, badge-medie, badge-mare	badge-hartie, badge-ebook, badge-audio	
cele trei elemente de test	Tema la Baze de date, Citit capitolul 3, Cumpărat cafea	Dune, Sapiens, 1984	

Observație: numele de fișiere, clasele CSS și identificatorii din cod se scriu fără diacritice (de exemplu `badge-hartie`, `etapa-01.md`); textele afișate utilizatorului se scriu cu diacritice.

Noțiuni introductive

Structura unei pagini HTML

O pagină HTML este un arbore de elemente. Elementele **semantice** descriu rolul fiecărei zone: `header` pentru antet, `main` pentru conținutul principal, `section` pentru o secțiune tematică, `footer` pentru subsol. Folosirea lor, în locul unor elemente `div` generice, face codul mai ușor de citit și pagina mai accesibilă. Pagina din această etapă are patru zone principale (figura 1.1).

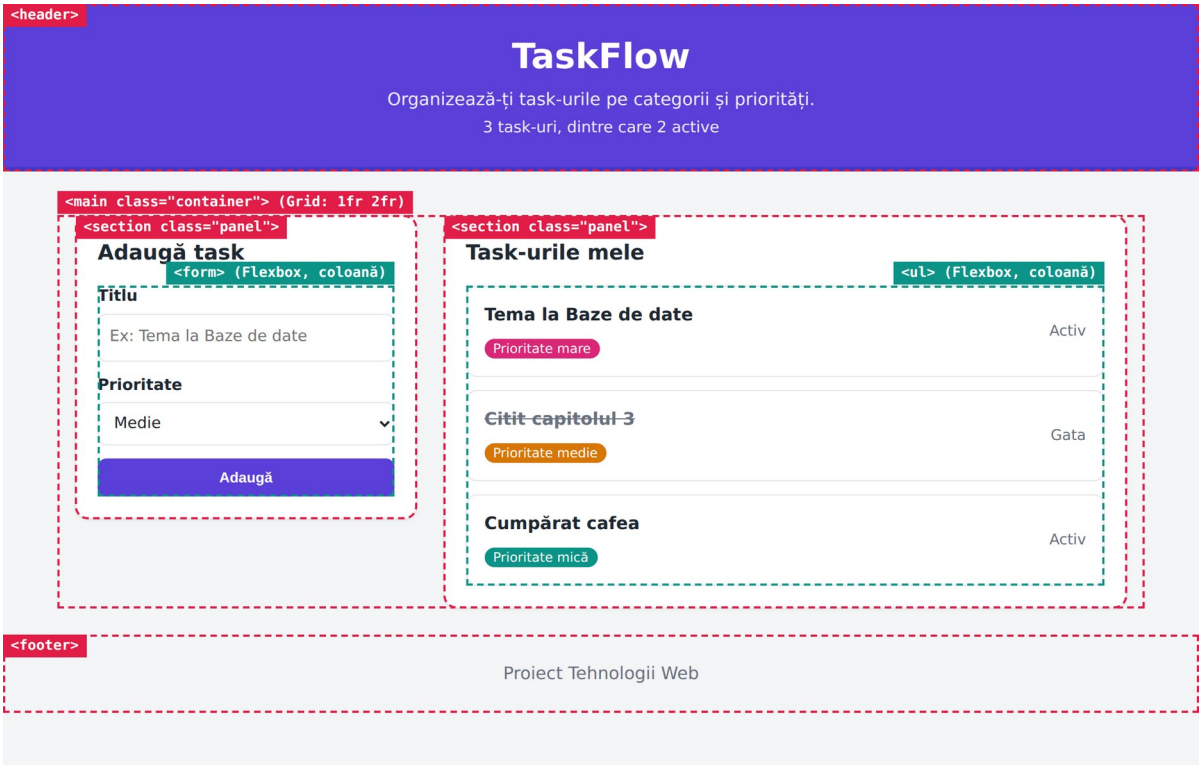


Fig. 1.1: Structura paginii. Cu roșu: zonele HTML; cu verde: elementele aranjate cu Flexbox

Modelul cutiei și box-sizing

Fiecare element ocupă în pagină o „cutie” formată din conținut, spațiere interioară (`padding`), bordură (`border`) și margine exterioară (`margin`). Implicit, lățimea declarată cu `width` se referă doar la conținut, iar padding-ul și bordura se adaugă peste ea. Regula `box-sizing: border-box` include padding-ul și bordura în lățimea declarată, astfel încât dimensiunile din layout devin previzibile.

Grid și Flexbox

Grid împarte un container în rânduri și coloane. În această etapă, containerul `main` are două coloane: formularul ocupă o fracțiune din lățime, iar lista două fracțiuni (`grid-template-columns: 1fr 2fr`).

Flexbox aranjează elementele pe o singură direcție, orizontală sau verticală. Se folosește în formular (etichetele și câmpurile, unele sub altele), în listă (cardurile, unele sub altele) și în fiecare card, unde titlul și starea sunt plasate la extremități (figura 1.2).

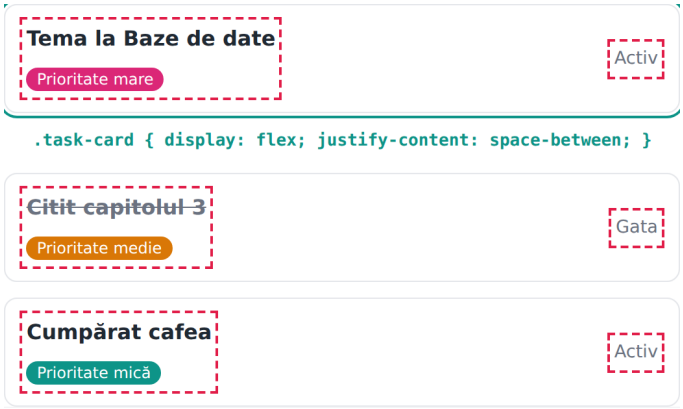


Fig. 1.2: Cele două componente ale fiecărui card, plasate la extremități cu justify-content: space-between

Media query și variabile CSS

O regulă `@media (max-width: 700px) { ... }` se aplică doar dacă fereastra are cel mult 700 px lățime. În această etapă, ea trece pagina pe o singură coloană (figura 1.3).

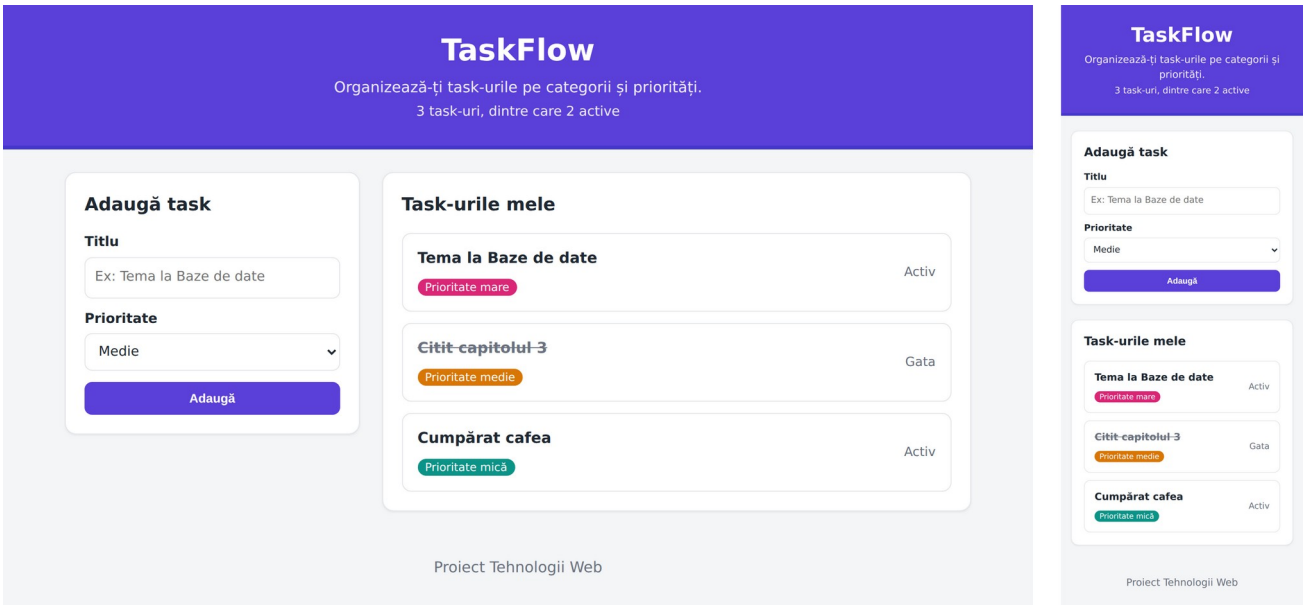


Fig. 1.3: Aceeași pagină pe desktop (stânga) și pe un ecran îngust (dreapta)

Variabilele CSS se definesc în `:root` (de exemplu `--accent: #5b3fd9;`) și se folosesc cu `var(--accent)`. Dacă toate culorile sunt definite prin variabile, tema întunecată se obține redefinind doar valorile variabilelor, fără a rescrie vreo regulă.

Git și GitHub

Git este un sistem de control al versiunilor: salvează, la cerere, starea fișierelor proiectului într-un **commit**, împreună cu un mesaj și data. **GitHub** este un serviciu care găzduiește online repository-urile Git. Fluxul de lucru folosit în proiect are trei comenzi: `git add` (alegerea fișierelor), `git commit` (salvarea versiunii local) și `git push` (publicarea pe GitHub).

Mediul de lucru

Sunt necesare:

- un editor de cod; se recomandă **Visual Studio Code**;
- **Git**; pe Windows se instalează Git for Windows, care include și terminalul *Git Bash*;
- un **cont GitHub** personal;
- un browser modern (Chrome, Edge sau Firefox), cu instrumentele pentru dezvoltatori (**DevTools**), deschise cu **F12**.

Comenzile din acest document sunt aceleași în terminalul din VS Code, în Git Bash și în PowerShell. Capturile de terminal au fost realizate pe un calculator cu utilizatorul *student*; pe calculatorul propriu, promptul și calea diferă.

Pasul A: Configurarea Git

La prima utilizare a Git pe un calculator, se configurează numele și adresa de e-mail care apar în commit-uri:

```
git --version
git config --global user.name "Prenume Nume"
git config --global user.email "adresa@exemplu.com"
```

Pasul B: Crearea folderului proiectului

Se creează un folder pentru proiect (în exemple, `proiect-tw`) și se deschide în VS Code (**File › Open Folder**). Toate fișierele din această etapă se creează în acest folder.

Desfășurarea lucrării

Pasul 1: Fișierul README.md și aprobarea temei

Fișierul `README.md` prezintă proiectul pe GitHub și ține loc de fișă a temei. Se actualizează la finalul fiecărei etape. Se recomandă redactarea lui în limba engleză, la fel ca întregul cod, pentru ca proiectul să poată fi inclus într-un portofoliu. README-ul conține: numele aplicației și o descriere de două rânduri, câmpurile unui element, cele trei elemente de test, modul de rulare și secțiunea *AI usage*.

Elementele de test se folosesc nemodificate în toate etapele; unul dintre ele trebuie să fie marcat ca finalizat. Categoriile și utilizatorul nu sunt încă afișate în pagină, dar se trec de pe acum în README. Șablonul de pornire (conținutul dintre < > se înlocuiește):

```
# <AppName>

<Two-line description: what the app manages and for whom.>

## Data model
| Field       | Type         | Notes                                |
| ----------- | ------------ | ------------------------------------ |
| <name>      | text         | required, max 100 chars              |
| <done flag> | boolean      | toggled from the list, default false |
| <fixed tag> | fixed values | <value1>, <value2>, <value3>         |
| <category>  | relation     | <Cat1>, <Cat2>, <Cat3>               |
| user        | relation     | the owner of the item (from week 11) |

Sample data used across all stages:
1. <item 1>, active, <tag>
2. <item 2>, done, <tag>
3. <item 3>, active, <tag>
```

```
## How to run
Open `index.html` in a browser. No build step, no server.

## AI usage
| Tool           | Used for                                  |
| -------------- | ----------------------------------------- |
| <e.g. ChatGPT> | <what exactly, e.g. CSS Grid, stage 1>    |
Details per stage: see the ai-log/ folder.

## Status
- [x] Stage 1: static mockup
□ Stage 2: data logic in JavaScript
```

După redactarea README-ului, tema se prezintă coordonatorului pentru aprobare. Numai după aprobare se continuă cu pașii următori.

Pasul 2: Repository-ul local și repository-ul GitHub

Pe GitHub se creează un repository nou (**New repository**), **public**, cu numele aplicației. Opțiunea *Add a README file* se lasă nebifată, deoarece fișierul există deja local. Apoi, în terminal, din folderul proiectului, se inițializează repository-ul local, se salvează README-ul și se leagă repository-ul local de cel de pe GitHub (figura 1.4):

```
git init
git add README.md
git commit -m "Add README with project theme"
git branch -M main
git remote add origin https://github.com/<utilizator>/<repo>.git
git push -u origin main
```

```
student@pc:~$ git --version
git version 2.43.0
student@pc:~$ git config --global user.name "Ion Popescu"
student@pc:~$ git config --global user.email "ion.popescu@stud.upb.ro"
student@pc:~$ cd proiect-tw
student@pc:~/proiect-tw$ git init
Initialized empty Git repository in /home/student/proiect-tw/.git/
student@pc:~/proiect-tw$ git add README.md
student@pc:~/proiect-tw$ git commit -m "Add README with project theme"
[main (root-commit) 992151a] Add README with project theme
 1 file changed, 31 insertions(+)
 create mode 100644 README.md
student@pc:~/proiect-tw$ git branch -M main
student@pc:~/proiect-tw$ git remote add origin https://github.com/ionpopescu/taskflow.git
student@pc:~/proiect-tw$ git remote -v
origin  https://github.com/ionpopescu/taskflow.git (fetch)
origin  https://github.com/ionpopescu/taskflow.git (push)
student@pc:~/proiect-tw$
```

Fig. 1.4: Inițializarea repository-ului, primul commit și legarea de GitHub

Comanda `git remote -v` afișează adresa repository-ului GitHub asociat. La primul `git push`, GitHub cere autentificarea. Parola contului nu mai este acceptată în terminal; autentificarea se face prin fereastra de browser deschisă de Git for Windows sau cu un *Personal Access Token*. După `git push`, la reîncărcarea paginii repository-ului, README-ul trebuie să fie vizibil.

Pasul 3: Structura HTML

În același folder se creează fișierele `index.html` și `style.css`. Pagina are patru zone (figura 1.1): antetul cu numele aplicației, o descriere și un contor de forma „3 elemente, dintre care 2 active”; containerul `main`, cu două elemente `section` (formularul de adăugare și lista); lista, formată dintr-un element `ul` cu trei carduri `li`; subsolul. Scheletul de pornire:

```html
<!DOCTYPE html>
<html lang="ro">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>NumeAplicatie - mockup</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header class="app-header">
    <h1>NumeAplicatie</h1>
    <p>Descriere scurtă.</p>
    <p class="app-count">3 elemente, dintre care 2 active</p>
  </header>
  <main class="container">
    <section class="panel">
      <h2>Adaugă ...</h2>
      <form class="item-form">
        <label>Titlu <input type="text" name="titlu" placeholder="Ex: ..." /></label>
        <label>Etichetă
          <select name="eticheta">
            <option value="v1">Valoare 1</option>
          </select>
        </label>
        <button type="submit">Adaugă</button>
      </form>
    </section>
    <section class="panel">
      <h2>Lista mea</h2>
      <!-- Când lista e goală: <p class="empty">Nu există niciun element.</p> -->
      <ul class="item-list">
        <li class="item-card">
          <div>
            <h3>Element 1</h3>
            <span class="badge badge-v1">Valoare 1</span>
          </div>
          <span class="status">Activ</span>
        </li>
        <li class="item-card done"> ... </li>
        <li class="item-card"> ... </li>
      </ul>
    </section>
  </main>
  <footer class="app-footer">Proiect Tehnologii Web</footer>
</body>
</html>
```

Contorul din antet va fi calculat automat în Etapa 3, iar comentariul cu `class="empty"` marchează locul mesajului pentru lista goală, afișat începând cu Etapa 4.

Fișierul se deschide în browser prin dublu clic sau prin tragerea lui în fereastra browserului. Fără stiluri, pagina afișează conținutul unul sub altul; după Pasul 4, arată ca în figura 1.5.

Pasul 4: Stilizarea CSS

Stilurile se scriu în `style.css`, în ordinea de mai jos. După fiecare secțiune, fișierul se salvează și pagina se reîncarcă în browser (**F5**).

4.1. Variabilele de culoare. Toate culorile se definesc o singură dată, în `:root`; în restul fișierului se folosește exclusiv `var(--nume)`:

```css
* { box-sizing: border-box; }

:root {
  --bg: #f4f5f7;        /* fundalul paginii */
```

```
  --card: #ffffff;      /* fundalul panourilor */
  --text: #1f2933;
  --muted: #6b7280;
  --accent: #5b3fd9;    /* culoarea de accent a aplicației */
  --accent-dark: #4338ca;
  --accent-soft: #ede9fe;
  --border: #e5e7eb;
  /* câte o culoare pentru fiecare valoare a etichetei */
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.5;
}
```

4.2. Antetul: fundal în culoarea de accent, text alb, centrat, cu spațiere interioară.

4.3. Cele două coloane (Grid). Containerul `main` are o lățime maximă, este centrat și se împarte în două coloane:

```
.container {
  max-width: 1000px;
  margin: 24px auto;
  padding: 0 16px;
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 24px;
  align-items: start;
}
```

4.4. Panourile: cele două elemente `section` primesc fundal `var(--card)`, bordură subțire, colțuri rotunjite și spațiere interioară.

4.5. Formularul (Flexbox): formularul și fiecare `label` primesc `display: flex` și `flex-direction: column`, cu distanța dintre elemente stabilită prin `gap`.

4.6. Cardul (Flexbox). Marcatorii listei se elimină (`list-style: none; padding: 0;` pe `ul`), iar fiecare card plasează titlul și starea la extremități:

```
.item-card {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 16px;
  border: 1px solid var(--border);
  border-radius: 10px;
}
```

4.7. Etichetele colorate: o clasă comună `.badge` (text mic, colțuri complet rotunjite, text alb) și câte o clasă pentru fiecare valoare, fiecare cu culoarea proprie.

4.8. Elementul finalizat: cardul cu clasa suplimentară `done` are titlul tăiat (`text-decoration: line-through`) și afișat în gri (`var(--muted)`).

4.9. Ecranele înguste:

```
@media (max-width: 700px) {
  .container { grid-template-columns: 1fr; }
}
```

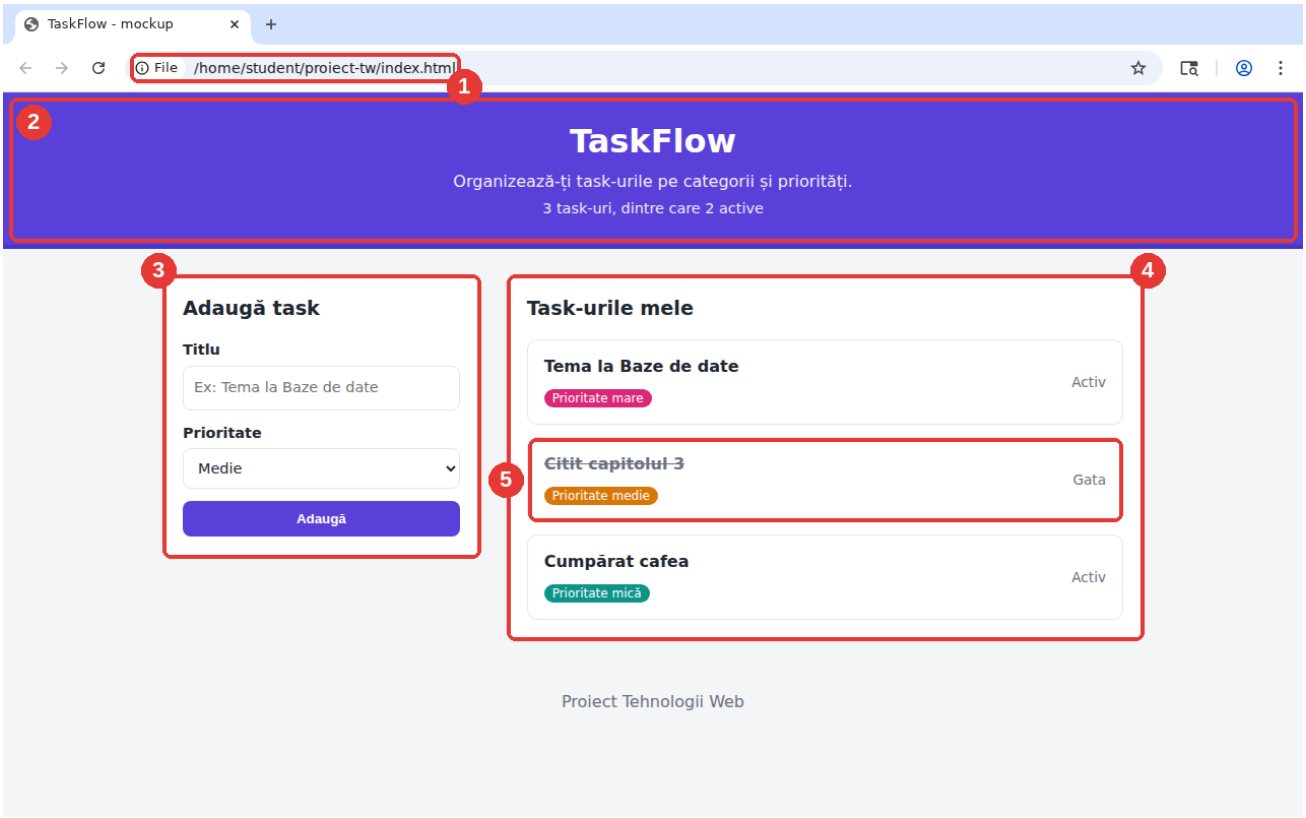


Fig. 1.5: Pagina deschisă în browser la finalul pasului 4

1. Calea fișierului deschis. Pagina se deschide direct de pe disc, fără server.
2. Antetul (`header`), cu numele aplicației, descrierea și contorul.
3. Panoul cu formularul, aranjat vertical cu Flexbox.
4. Panoul cu lista: cele trei carduri cu datele de test.
5. Cardul marcat ca finalizat (clasa `done`): titlul tăiat și afișat în gri.

Pasul 5: Stările interactive și tema întunecată

Se adaugă un efect `:hover` pe card (de exemplu bordura în culoarea de accent și un fundal ușor colorat) și pe buton. Se adaugă apoi un contur vizibil pentru orice element selectat cu tasta **Tab**, necesar utilizatorilor care navighează de la tastatură (figura 1.6):

```
:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
}
```

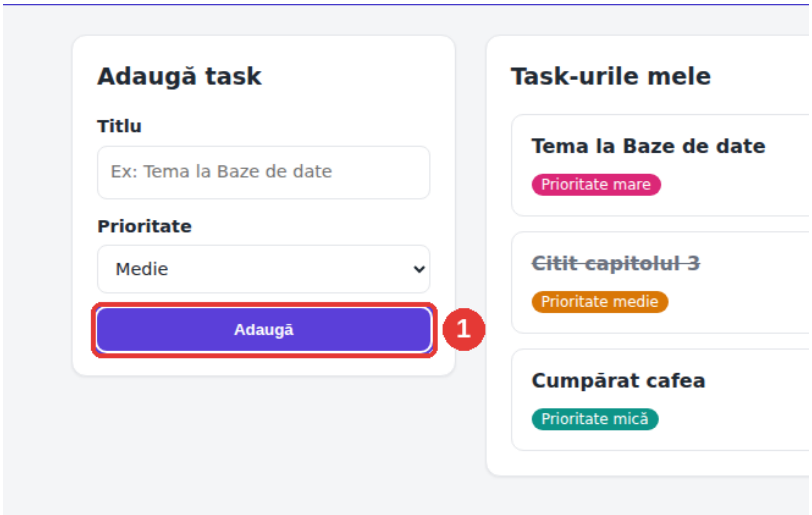


Fig. 1.6: Conturul de focus pe butonul Adaugă, după navigarea cu tasta Tab

1. Elementul selectat cu tasta Tab este încadrat de conturul definit prin `:focus-visible`.

Tema întunecată redefinește doar valorile variabilelor, pentru cazul în care sistemul utilizatorului folosește modul întunecat:

```
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #14121d;
    --card: #1e1b2e;
    --text: #e5e7eb;
    --muted: #9ca3af;
    --border: #352f4d;
    --accent-soft: #2a2342;
  }
}
```

Dacă în tema întunecată rămân zone albe sau text greu lizibil, cauza este o culoare scrisă direct, fără variabilă; corectura se face în regula respectivă.

Pasul 6: Verificarea în DevTools

Ecranele înguste. Se deschide DevTools (**F12**) și se activează modul pentru dispozitive mobile cu **Ctrl + Shift + M** (pe Mac, **Cmd + Shift + M**).

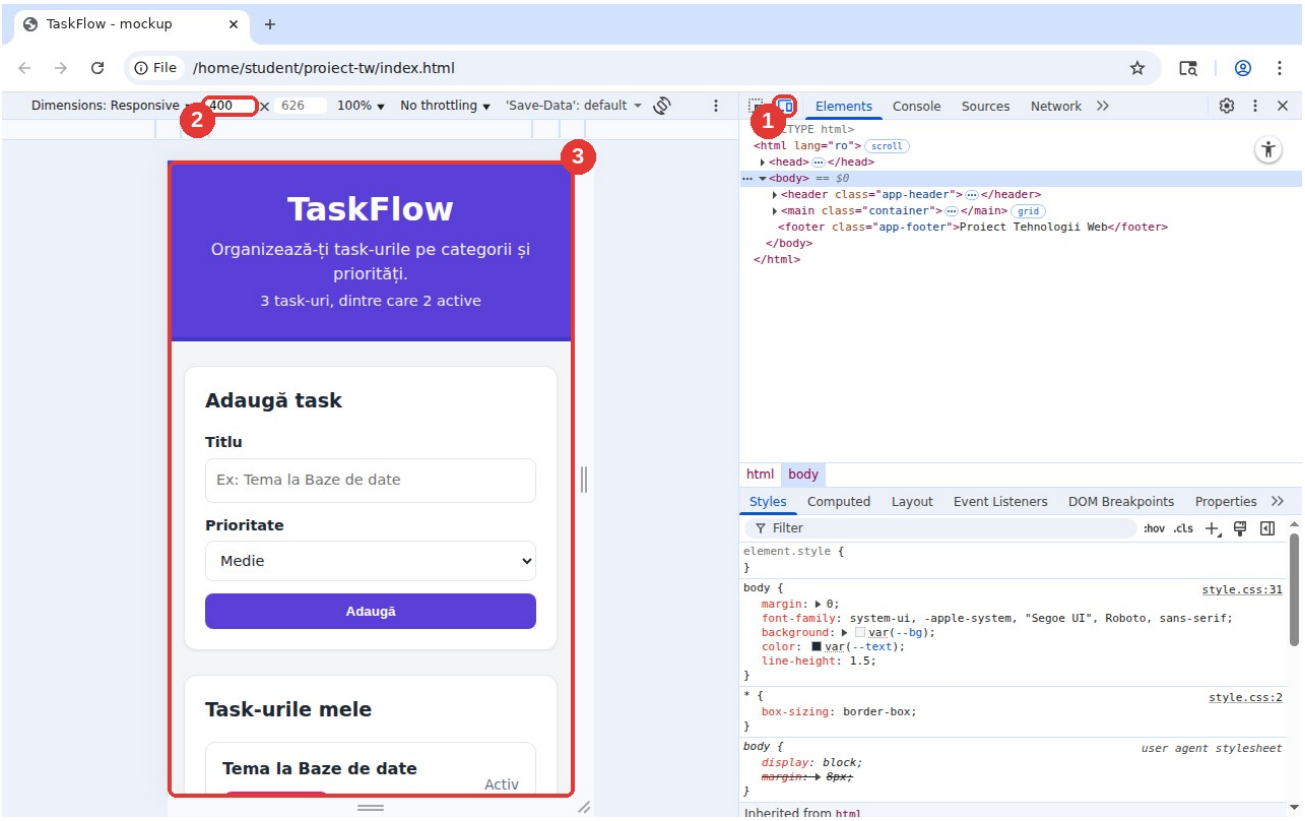


Fig. 1.7: Modul pentru dispozitive mobile din DevTools

1. Butonul *Toggle device toolbar*, echivalent cu Ctrl + Shift + M.
2. Lățimea simulată a ecranului. Se introduce o valoare sub 700 (aici 400).
3. Pagina trece pe o singură coloană: formularul se afișează deasupra listei.

Tema întunecată se verifică fără modificarea setărilor sistemului. În DevTools se apasă **Ctrl + Shift + P**, se scrie *Show Rendering* și se apasă Enter. În panoul *Rendering* deschis în partea de jos, opțiunea *Emulate CSS media feature prefers-color-scheme* se setează pe **prefers-color-scheme: dark** (figura 1.8).

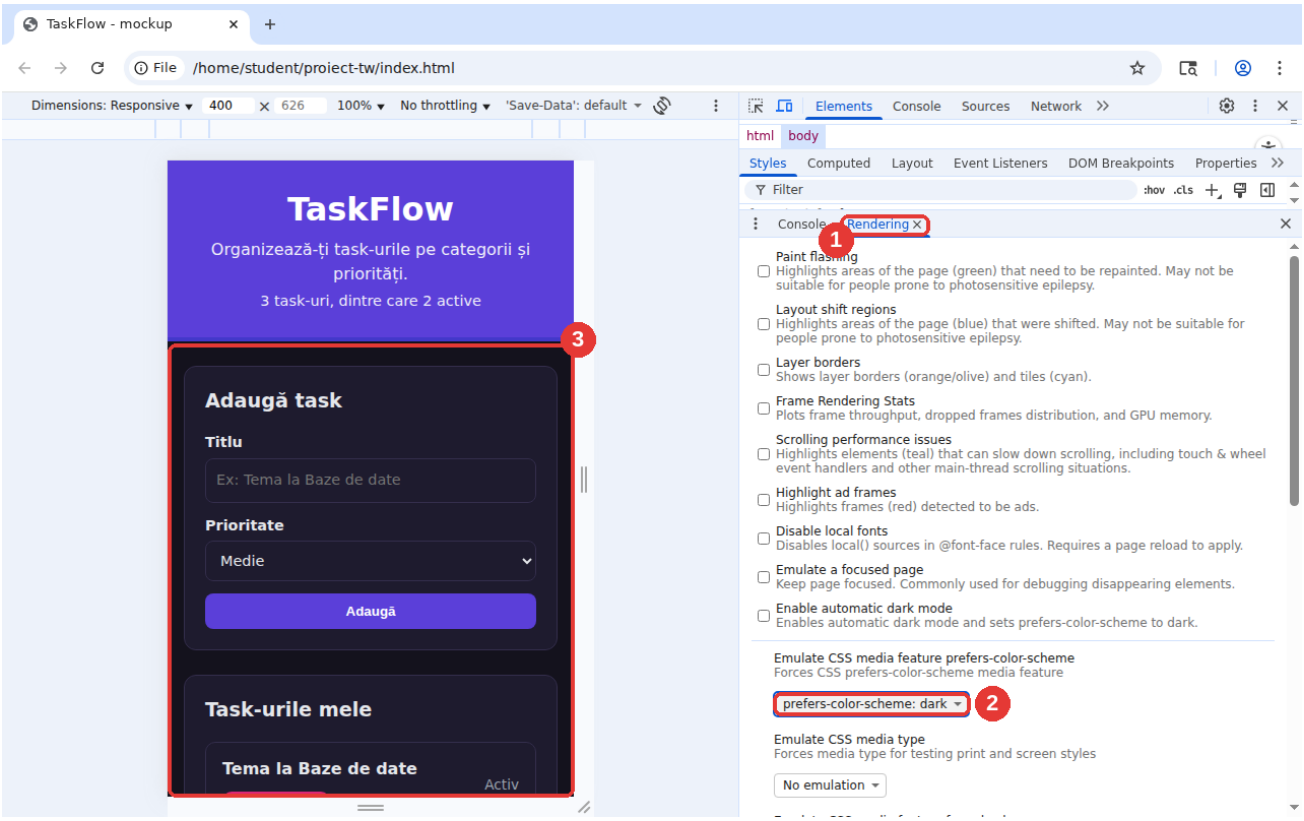


Fig. 1.8: Emularea temei întunecate din panoul Rendering

1. Panoul *Rendering*.
2. Opțiunea *prefers-color-scheme: dark*.
3. Pagina afișată cu variabilele temei întunecate.

Navigarea cu tastatura. Se face clic într-o zonă goală a paginii și se apasă de mai multe ori tasta **Tab**: fiecare câmp și butonul trebuie să primească, pe rând, conturul din figura 1.6.

Pasul 7: Jurnalul AI și publicarea pe GitHub

Se creează folderul `ai-log` și, în acesta, fișierul `etapa-01.md`, completat după șablonul de mai jos (în limba engleză):

```
# Stage 1: AI log

## Tools
- <e.g. ChatGPT, Copilot in VS Code>

## Conversations
- <share link>  (<what it was about, e.g. CSS Grid layout>)

## Key requests
### 1. <short title>
- Asked: <what I asked>
- Got: <what the answer suggested>
- Changed or rejected: <what I did differently and why>

## What I learned / what did not work
<3-4 lines>
```

Dacă în această etapă nu s-a folosit niciun instrument AI, fișierul conține doar: `No AI tools were used in this stage.` Apoi, toate fișierele se publică pe GitHub (figura 1.9):

```
git add .
git commit -m "Stage 1: static mockup with HTML and CSS"
git push
```

```
student@pc:~/proiect-tw$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
        style.css

nothing added to commit but untracked files present (use "git add" to track)
student@pc:~/proiect-tw$ git add .
student@pc:~/proiect-tw$ git commit -m "Stage 1: static mockup with HTML and CSS"
[main 1e6f998] Stage 1: static mockup with HTML and CSS
 2 files changed, 245 insertions(+)
 create mode 100644 index.html
 create mode 100644 style.css
student@pc:~/proiect-tw$ git log --oneline
1e6f998 (HEAD -> main) Stage 1: static mockup with HTML and CSS
992151a Add README with project theme
student@pc:~/proiect-tw$
```

Fig. 1.9: Starea repository-ului înainte de commit, commit-ul etapei și istoricul

Comanda `git status` arată fișierele noi, încă neincluse într-un commit (cu roșu). După `git add .` și `git commit`, `git log --oneline` afișează istoricul: cel mai recent commit apare primul.

Tabelul de verificare din README

La finalul fiecărei etape, README-ul conține un tabel cu toate cerințele etapei, identificate prin codul din lista de verificare. Pentru fiecare cerință, coloana *Where* conține un **permalink** GitHub, adică un link fix către anumite linii de cod dintr-un commit publicat, iar coloana *How to check* precizează pe scurt cum se verifică. Tabelul servește atât ca listă de control pentru student, cât și ca punct de plecare pentru verificarea făcută de coordonatori.

Un permalink se obține astfel:

1. după `git push`, se deschide fișierul pe GitHub;
2. se face clic pe numărul primei linii din interval, apoi Shift + clic pe numărul ultimei linii;
3. din meniul **…** afișat lângă linii se alege **Copy permalink** (alternativ, tasta **Y** transformă adresa din bara browserului într-un permalink).

Deoarece permalink-urile se referă la un commit deja publicat, tabelul se completează după publicarea codului etapei și se publică într-un commit separat, `Stage 1: checklist`. Conținutul tabelului pentru această etapă:

Tabelul 1.4: Tabelul de verificare al Etapei 1

ID	Requirement	Where (permalink)	How to check
S1-R1	README: description, fields, sample data, how to run	README.md	read
S1-R2	AI usage section	README.md	read
S1-R3	AI log for stage 1	ai-log/etapa-01.md	read
S1-R4	header, form (text + select), 3 cards with own data	index.html#L..-L..	open the page
S1-R5	finished card looks different	style.css#L.. (.done)	look at the card
S1-R6	2 columns on desktop, 1 under 700px	style.css#L.. (@media)	resize < 700px
S1-R7	visible focus, readable dark theme	style.css#L..	Tab; dark mode
S1-R8	commit “Stage 1” pushed	link to the commit	commit history

În README, tabelul se scrie în Markdown, iar în coloana *Where* se pune link-ul, de exemplu: `[index.html#L12-L48](https://github.com/<user>/<repo>/blob/<commit>/index.html#L12-L48)`. O cerință pentru care nu se poate indica un loc în cod este, cel mai probabil, neîndeplinită.

Lista de verificare a etapei

Lista se parcurge înainte de verificarea cu coordonatorul. La verificare se prezintă pagina deschisă în browser și repository-ul GitHub.

☐ **S1-R1** Tema este aprobată, iar README-ul conține descrierea, câmpurile, elementele de test și modul de rulare.

☐ **S1-R2** README-ul conține secțiunea *AI usage* (sau mențiunea că nu s-au utilizat instrumente AI).

☐ **S1-R3** Fișierul `ai-log/etapa-01.md` este completat conform șablonului.

☐ **S1-R4** Pagina conține antetul, formularul cu câmpul text și lista fixă, precum și trei carduri cu datele temei.

☐ **S1-R5** Un card este marcat vizibil ca finalizat.

☐ **S1-R6** Pe desktop se afișează două coloane, iar sub 700 px o singură coloană.

☐ **S1-R7** Focusul este vizibil la navigarea cu Tab, iar tema întunecată este lizibilă.

☐ **S1-R8** Codul este publicat pe GitHub, într-un repository public, cu commit-ul `Stage 1: ...`.

Criterii de calitate urmărite, pe lângă conformitatea cu figura 1.5: toate culorile sunt definite prin variabile, iar tema întunecată funcționează fără reguli duplicate; cardurile și butonul reacționează la cursor; marcajul pentru lista goală (clasa `.empty`) este pregătit; pagina folosește elemente semantice, nu exclusiv `div`.

Probleme frecvente

Tabelul 1.5: Probleme frecvente și rezolvarea lor

Simptom	Cauză probabilă	Rezolvare
stilurile nu se aplică	calea din `<link>` este greșită sau `style.css` nu este în același folder	verificarea numelui și a locului fișierului
pe telefon se păstrează două coloane	lipsește `<meta name="viewport">` sau `@media` este plasat înaintea regulii `.container`	adăugarea meta-etichetei; mutarea regulii `@media` la final
în tema întunecată rămân zone albe	o culoare este scrisă direct, nu prin `var(--...)`	înlocuirea culorii cu variabila corespunzătoare
cardurile au marcatori în stânga	lipsesc `list-style: none` și `padding: 0` pe `ul`	adăugarea celor două proprietăți
`git push` cere parola și eșuează	GitHub nu mai acceptă parola contului	autentificare prin browser sau cu un Personal Access Token
diacriticele se afișează incorect	lipsește `<meta charset="UTF-8" />`	adăugarea meta-etichetei în `head`

Cerințe până la etapa următoare

Pașii nefinalizați în timpul ședinței se completează individual. Până la etapa următoare, tema trebuie să fie aprobată, iar codul, README-ul (cu tabelul de verificare) și jurnalul `ai-log/etapa-01.md` trebuie să fie publicate pe GitHub. În Etapa 2 se implementează primele funcții JavaScript care operează pe datele temei; se recomandă recitirea README-ului și stabilirea valorilor posibile pentru fiecare câmp.